SRAM-Based Digital CIM Macro for Linear Interpolation and MAC

Zhiting Lin, Yunlong Liu, Yaling Wang, Yue Zhao, Chunyu Peng, Xiulong Wu*
Anhui University, Hefei, China
**Corresponding Author Email: xiulong@ahu.edu.cn*

2024 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-3099-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISCAS58744.2024.10558525

***Abstract*—Linear interpolation is widely used in algorithms such as image segmentation, but the existing compute-in-memory (CIM) architectures cannot satisfy the needs of linear interpolation. This paper proposes a CIM macro based on static random-access memory (SRAM) that implements linear interpolation for the first time. Swing multiplication and accumulation are proposed in this paper for linear interpolation operations. In addition, the proposed circuit can be used for multiply-and-accumulate (MAC) operation and supports parallel updating and computing. A new adder tree by reused the adders inside the multiplier to implement the accumulation operation. The proposed circuit can improve the area efficiency as no additional adder tree circuit is required. The design is implemented in a 28 nm process and area efficiency can achieve 0.059 TOPS/mm² for MAC operation. When operating with an 8-bit linear interpolation, this CIM macro achieves access times of 6−9 ns and energy efficiencies of 11.13−17.72 TOPS/W. Under MAC operation with 8-bit inputs and weights, this CIM macro achieves access time of 6.36−9.47 ns and energy efficiency of 4.61−8.36 TOPS/W for 19-bit outputs.**

***Keywords*—SRAM, Compute-in-memory (CIM), Multiply-and-accumulate (MAC), Linear interpolation, Adder tree**

I. INTRODUCTION

Artificial intelligence and machine learning are used in various fields, and their associated algorithms require large amounts of data processing. However, in the traditional Von Neumann architecture shown in Fig. 1(a), the computation and storage modules are separated. A large amount of data should be transferred repeatedly between the computation and storage units, resulting in significant delays and energy consumption [1]. Compute-in-memory (CIM) architecture is an emerging paradigm that addresses the bottleneck of the Von Neumann architecture. As shown in Fig. 1(b), the concept of the CIM architecture is to achieve the integration of storage and computation units and complete the computation inside the memory array [2].

A schematic diagram of image segmentation using the U-Net network is shown in Fig. 2. After the input feature map is convolved and down-sampled to extract the features, it should be convolved and up-sampled to enlarge the features. Existing CIMs can perform MAC efficiently, but do not work well for linear interpolation. [5] [6] [7] [8].

CIM can be classified into analog and digital methods. Although analog methods are highly energy efficient, the accuracy of the computed results depends on the process, voltage, and temperature. By contrast, digital CIM methods have high computational accuracy and robustness. Two research

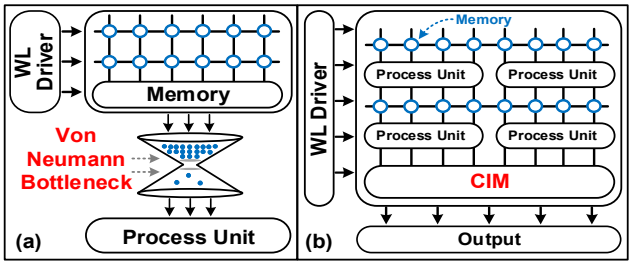

Fig. 1. (a) Traditional Von Neumann architecture and (b) schematic of the CIM architecture.

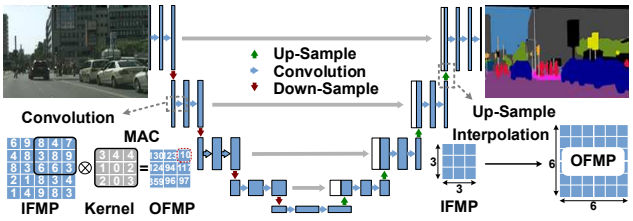

Fig. 2. Schematic diagram of U-Net structure for image segmentation [3] [4].

directions currently exist for CIM that use digital methods. The first is to connect a single random-access memory (SRAM) cell tightly to a logic cell such that every data point participates in the logic operation. However, this approach performs only single-bit operations. To implement multi-bit operations, additional circuits are required to expand the number of operation bits [9] [10]. The second direction is to integrate the logic cell next to the SRAM array. The data stored in the SRAM array are read to the bit line (BL), and the logic cell is connected to the BL. Compared to the combination of a single SRAM cell and a logic cell, this approach has better area utilization as the logical cells are reused [11] [12].

Digital methods use adder trees to accumulate the partial results into a final result. However, the adder tree not only occupies a large area and consumes a large amount of power, but also is idle during multiplication operations, undermining the energy efficiency of digital CIM methods [13] [14].

To overcome these challenges, this article proposes, for the first time, a fully digital CIM architecture that enables both linear interpolation and MAC with good energy efficiency and area utilization. The advantages of the proposed architecture are as follows. (1) A reconfigurable multiplier that performs both MAC or linear interpolation operations efficiently in a single circuit. (2) A reusable adder tree is proposed, where the adders

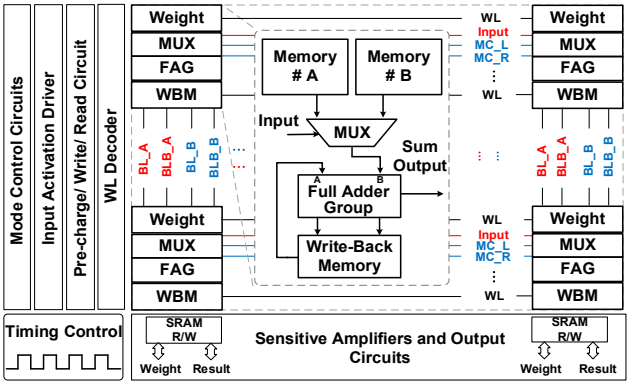

Fig. 3. Overall CIM architecture suitable for linear interpolation and MAC operations.

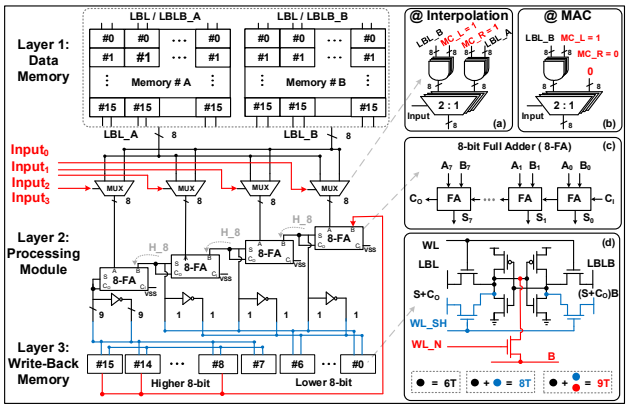

Fig. 4. The circuit structure of the multiplier. Mode-control signals for (a) interpolation mode and (b) MAC mode. (c) 8-bit Full Adder. (d) 8T, 9T cell structure for write-back memory.

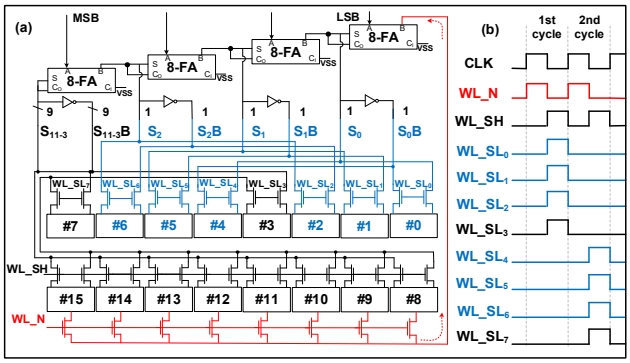

Fig. 5. (a) The circuits between the processing module and the write-back memory. (b) The control timing for write-back memory.

in the multiplier are reused to construct the adder tree. (3) This architecture supports parallel weight updating and multiplication computation to shorten the latency.

The remainder of this paper is organized as follows. Section II describes the proposed CIM architecture. Section III provides an analysis of the simulation results. Finally, Section IV concludes the paper.

II. Proposed Architecture

A. Characterization of the Linear Interpolation

The purpose of linear interpolation is to find an intermediate coordinate between the two coordinates. For example, when the coordinates (X_0, Y_0) and (X_1, Y_1) are known, to obtain intermediate coordinates (X, Y), the following equation can be utilized:

$$\frac{Y - Y_0}{X - X_0} = \frac{Y_1 - Y_0}{X_1 - X_0} \tag{1}$$

By simplifying equation (1), the following equation is obtained:

$$\begin{aligned} Y &= \frac{X - X_0}{X_1 - X_0} Y_1 + \frac{X_1 - X}{X_1 - X_0} Y_0 \\ &= \alpha Y_1 + (1 - \alpha) Y_0 \\ &= \alpha A + (1 - \alpha) B \end{aligned} \tag{2}$$

X in equation (2) is known and is used to set the position of the intermediate coordinate. To avoid the excessive use of subscripts, we denote Y_1 as A and Y_0 as B.

The linear Interpolation coefficients α and 1–α are between 0 and 1, which can be expressed as a binary fixed-point number with an 8-bit fractional part. The coefficients α and 1–α are inverses in each of the 8-bit fractional parts. For example, when "α = 0.1010 1010," then "1 – α = 0.0101 0101." We name the 8-bit fractional parts of α as α_7, α_6, α_5, α_4, α_3, α_2, α_1, and α_0. Taking advantage of this special relationship between α and 1–α, datum A or B is selected by judging whether α_i is 0 or 1, and the result of $A\alpha + B(1–\alpha)$ can be obtained.

B. Overall Architecture and Proposed Circuit

To achieve low-power, high-precision linear interpolation and MAC, we propose a novel multiplication and accumulation architecture. The overall CIM framework is shown in Fig. 3 and consists of 136 rows × 128 columns SRAM arrays and processing modules. The SRAM arrays and processing modules are divided into 64 blocks, each containing two 16 × 8 SRAM arrays, which are labeled as memories #*A* and #*B*, MUX, a full adder group (FAG) for computing operations, and a 1 row × 16 columns SRAM array for write-back operations, which are labeled as write-back memory (WBM). Each block is used as a multiplier.

As shown in Fig. 4, the proposed multiplier has the following two modes. In linear interpolation mode, this circuit can perform two 8-bit multiplication operations and one 16-bit addition operation. In MAC mode, this circuit can perform 8-bit MAC operation. Under conventional SRAM operation, the WBM functions in the same manner as other SRAM cells. However, during computational operation, the WBM is used to store the results. The multiplier circuit can be divided into three layers.

Layer 1: Data Memory. The memory consisting of 16 rows and 8 columns is used to store the coordinates A and B for linear interpolation, or weights for MAC. The eight pairs of local bit lines (LBL) in memory #*A* are called LBL_A, and the eight pairs of LBLs in memory #*B* are called LBL_B.

Layer 2: Processing Module. Each of the four MUXs represents a different weight. The bit-serial method is used to divide the 8-bit input data into two cycles. The 4-bit input data of each cycle are input in parallel and are utilized as selector signals for the MUX. As shown in Figs. 4(a) and (b), LBL_A, LBL_B, and the mode-control signals MC_L and MC_R are connected to the AND logic gate circuits to switch Interpolation

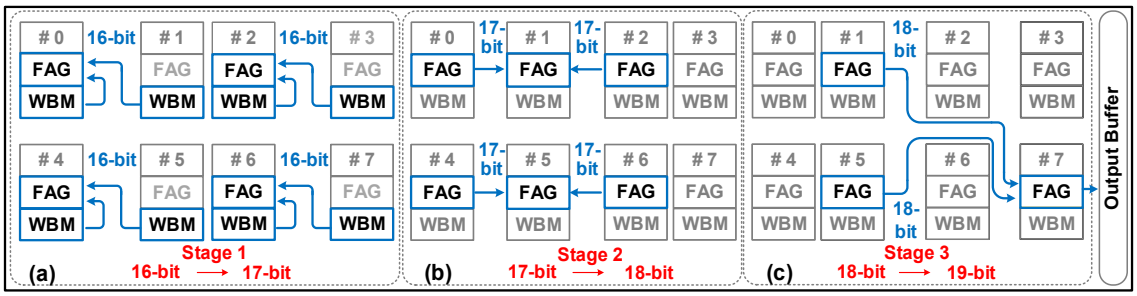

Fig. 9. FAGs within the multipliers of the eight blocks are grouped to form the adder tree.

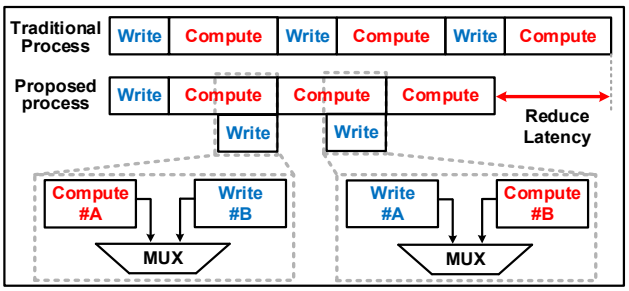

Fig. 6. Conventional process and proposed process that can reduce latency.

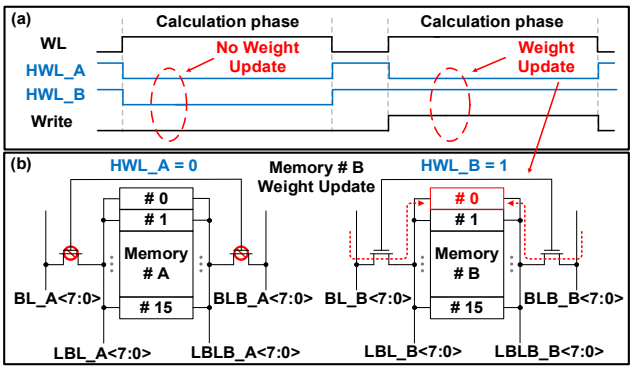

Fig. 7. Weight update during computing.

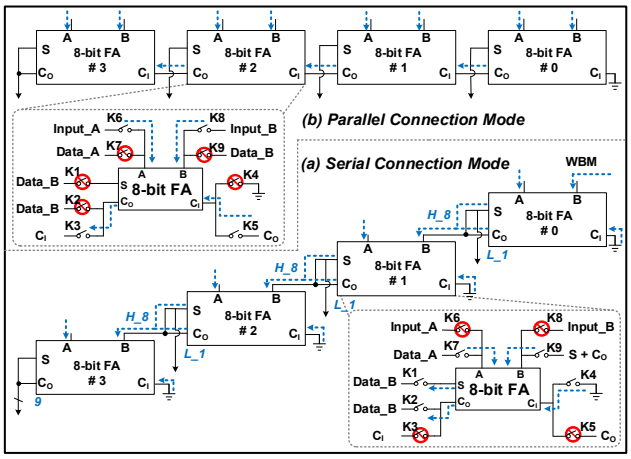

Fig. 8. Mode conversion of FA group.

mode and MAC mode. Four 8-FAs are used in this multiplier to form a serial adder, where each 8-FA is composed of eight 1-bit FAs connected serially, as shown in Fig. 4(c).

Level 3: Write-back memory. As shown in Fig. 4(d), the WBM consists of both 8T and 9T SRAM cells. Because the bit-serial method is used, the results of the first cycle should be saved and employed in the computation of the next cycle. A pair of NMOS transistors was added to the 6T SRAM memory controlled by the local word line (WL) called WL_SH. An NMOS transistor controlled by the local WL called WL_N was added to the connection path between the write-back memory and the input port of the 8-FA to separate the computation and write operations. As shown in Fig. 5(a) is a detailed circuit between the processing module and the write-back memory. Fig. 5(b) shows the control timing of 8T and 9T in write-back memory.

C. *Parallel Update and Computation*

The multiplier described in II. B uses MUX to enable memories #A and #B to update the weights and calculations alternately. As shown in Fig. 6, when memory *#A* performs computation, memory *#B* can perform a write operation, and vice versa.

Fig. 7(a) depicts the control timing of two different computation phases: one without updating the weight, and the other with updating the weight in memory *#B*. Fig. 7(b) demonstrates that memory *#A* is involved in the multiplication computation and memory *#B* remains idle. The horizontal read/write word line (HWL_A) of *#A* is set to a low voltage and the horizontal read/write word line (HWL_B) of *#B* is set to a high voltage. The new weight is transported from the global BLs called BL_B to the local BLs to update the weight of memory *#B*.

D. *Adder Tree*

As shown in Fig. 8, the proposed FA group can be converted between the serial and parallel connection modes. The FA group in the parallel connection mode can implement an adder tree for the accumulation operation. Switches K6, K7, K8, and K9 exist at the A and B ports of the 8-bit FA; K4 and K5 at the C_I port; switches K2 and K3 at the C_O port; and switch K1 at the S port. As shown in Fig. 8(a), when the FA group is serially connected, switches K1, K2, K4, K7, and K9 are turned on. As shown in Fig. 8(b), switches K3, K5, K6, and K8 are turned on when the FA group is connected in parallel.

As shown in Fig. 9, the new adder tree proposed in this paper accumulates the multiplication results of the weight and input in eight blocks, numbered from #7 to #0. The adders utilized in this adder tree reuse the FA group in each block. The accumulation process is divided into three stages.

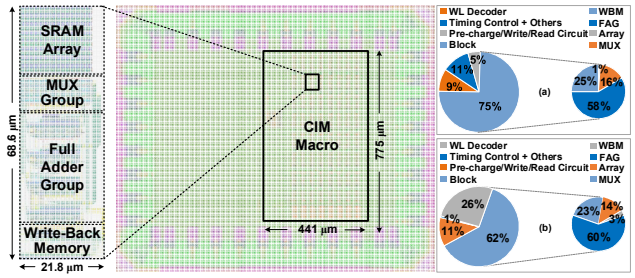

Fig. 10. Block layout (left), complete layout (center), and (a) power/ (b) area breakdown (right).

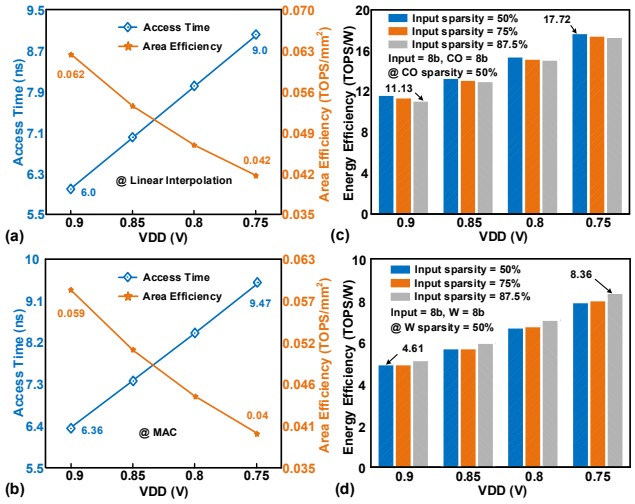

Fig. 11. Access time and area efficiency for (a) linear interpolation and (b) MAC at different supply voltages. Energy efficiency for (c) linear interpolation and (d) MAC at different supply voltages and sparsities.

III. Evaluation Results

The proposed design was implemented in a 136×128 SRAM array using a 28 nm CMOS process. The complete and block layout is shown in Fig. 10, where CIM macro occupies 0.3418 mm^2 area. In addition, Fig. 10(a) and (b) show the power and area breakdown of the CIM macro, respectively. Parasitic parameters were extracted from the layout and employed in the post-simulation. In this section, the simulation results for the performance of the proposed circuit are analyzed and compared with previous works.

As shown in Fig. 11(a) are the access time and area efficiency of linear interpolation at different supply voltages. From 0.75 V to 0.9 V supply voltage, the minimum access time is 6 ns and the maximum area efficiency is 0.062 TOPS/mm^2. Access time and area efficiency of MAC at different supply voltages are shown in Fig. 11(b). From 0.75 V to 0.9 V supply voltage, the minimum access time is 6.36 ns and the maximum area efficiency is 0.059 TOPS/mm^2.

Fig. 11(c) shows the energy efficiency of linear interpolation at 50% coordinate sparsity, where CO represents the coordinate. The energy efficiency is 17.72 TOPS/W for 0.75 V at 50% input sparsity. Fig. 11(d) depicts the energy efficiency of MAC at 50% weight sparsity, where W represents the weight. The energy efficiency at 87.5% input sparsity can reach 8.36 TOPS/W with 0.75V supply voltage.

TABLE I. Comparison to Previous Works

Reference	TCAS-I 2023 [17]	TCAS-II 2023 [15]	JSSC 2021 [9]	JSSC 2022 [16]	This work
Technology	65 nm	180 nm	65 nm	28 nm	**28 nm**
Memory	SRAM	SRAM	SRAM	SRAM	**SRAM**
Voltage (V)	0.76 – 1.2	1 – 1.8	0.6 – 0.8	0.85 – 1	**0.75 – 0.9**
Access Time (ns)	3	36	7.2	17.2	**9.47 – 6.36**
Array Size	101 kb	16 kb	16 kb	64 kb	**17 kb**
Macro Area(mm^2)	2.157	3.41	0.2272	N/A	**0.3418**
8b Area Eff. (TOPS/mm^2)	0.0199	0.0029	0.1054	N/A	**0.059** [4, 6]
Input Precision	8	8	1 – 16	8	**8**
Weight Precision	8	8	1 – 16	8	**8**
Output Precision	8	22	1 – 16	20	**19**
8b Energy Eff. (TOPS/W)	3.53 [2]	3.8 [2]	6.87 [2]	7.6 [2]	**8.36 – 4.61** [1, 4, 5]
Simultaneous MAC + Write	No	No	No	No	**Yes**

[1] Post-layout simulation.
[2] Chip measurement.
[3] The input and weight are normalized to 8-bit.
[4] 2 Operation (OP) = 1multiplication + 1 accumulation.
[5] Simulation at 50% weight sparsity.
[6] Consider CIM macro area, exclude I/O Interface and other peripheral.

The performances of the previous methods and the CIM architecture proposed in this paper for MAC operations are compared in Table I. The energy efficiency of the MAC can reach 8.36 TOPS/W. Because the adder tree is composed of reusing circuits without additional area overhead, the area efficiency is better and can reach 0.059 TOPS/mm^2. In conclusion, the proposed architecture provides higher energy efficiency and better area efficiency compared to existing CIM architectures.

IV. Conclusion

This paper proposes an SRAM-based CIM architecture that implements linear interpolation for the first time. The swing multiplication operations are implemented using MUX, which is well-suited for linear interpolation. The proposed circuit can also be used for MAC and supports parallel updating and computing. Further, a new adder tree was proposed for the accumulation by reusing the adder inside the multiplier. The design is implemented in a 28 nm process and area efficiency can achieve 0.059 TOPS/mm^2 for MAC operation. When operating with an 8-bit linear interpolation, this CIM macro achieves access times of 6−9 ns and energy efficiencies of 11.13−17.72 TOPS/W. Under MAC operation with 8-bit inputs and weights, this CIM macro achieves access time of 6.36−9.47 ns and energy efficiency of 4.61−8.36 TOPS/W for 19-bit outputs.

Acknowledgment

This work was supported in part by the University Synergy Innovation Program of Anhui Province No. GXXT-2023-013; the National Natural Science Foundation of China, under Grant No. 62074001; the State Key Program of the National Natural Science Foundation of China, under Grant No. 61934005; the National Key R&D Program of China, under Grant No. 2018YFB2202602; and the Joint Funds of the National Natural Science Foundation of China, under Grant No. U19A2074.

REFERENCES

[1] X. Si *et al.*, "A Twin-8T SRAM Computation-in-Memory Unit-Macro for Multibit CNN-Based AI Edge Processors," *IEEE Journal of Solid-State Circuits,* vol. 55, no. 1, pp. 189-202, 2020.

[2] Z. Jintao, W. Zhuo, and N. Verma, "A machine-learning classifier implemented in a standard 6T SRAM array," in *2016 IEEE Symposium on VLSI Circuits (VLSI-Circuits)*, 15-17 June 2016 2016, pp. 1-2.

[3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, Cham, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds., 2015// 2015: Springer International Publishing, pp. 234-241.

[4] A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollár, "Panoptic Segmentation," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 15-20 June 2019 2019, pp. 9396-9405.

[5] K. Ando *et al.*, "BRein Memory: A Single-Chip Binary/Ternary Reconfigurable in-Memory Deep Neural Network Accelerator Achieving 1.4 TOPS at 0.6 W," *IEEE Journal of Solid-State Circuits,* vol. 53, no. 4, pp. 983-994, 2018.

[6] S. Yin, Z. Jiang, J. S. Seo, and M. Seok, "XNOR-SRAM: In-Memory Computing SRAM Macro for Binary/Ternary Deep Neural Networks," *IEEE Journal of Solid-State Circuits,* vol. 55, no. 6, pp. 1733-1743, 2020.

[7] S. Lee et al., "A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, 2022, pp. 1-3.

[8] J. D. Ferreira et al., "pLUTo: Enabling Massively Parallel Computation in DRAM via Lookup Tables," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 900-919.

[9] H. Kim, T. Yoo, T. T. H. Kim, and B. Kim, "Colonnade: A Reconfigurable SRAM-Based Digital Bit-Serial Compute-In-Memory Macro for Processing Neural Networks," *IEEE Journal of Solid-State Circuits,* vol. 56, no. 7, pp. 2221-2233, 2021.

[10] A. Agrawal, A. Jaiswal, C. Lee, and K. Roy, "X-SRAM: Enabling In-Memory Boolean Computations in CMOS Static Random Access Memories," *IEEE Transactions on Circuits and Systems I: Regular Papers,* vol. 65, no. 12, pp. 4219-4232, 2018.

[11] J. Wang *et al.*, "A 28-nm Compute SRAM With Bit-Serial Logic/Arithmetic Operations for Programmable In-Memory Vector Computing," *IEEE Journal of Solid-State Circuits,* vol. 55, no. 1, pp. 76-86, 2020.

[12] H. Jia, H. Valavi, Y. Tang, J. Zhang, and N. Verma, "A Programmable Heterogeneous Microprocessor Based on Bit-Scalable In-Memory Computing," *IEEE Journal of Solid-State Circuits,* vol. 55, no. 9, pp. 2609-2621, 2020.

[13] B. Yan et al., "A 1.041-Mb/mm2 27.38-TOPS/W Signed-INT8 Dynamic Logic-Based ADC-less SRAM Compute-in-Memory Macro in 28nm with Reconfigurable Bitwise Operation for AI and Embedded Applications," in *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, 20-26 Feb. 2022 2022, vol. 65, pp. 188-190.

[14] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm2 Fully-Digital Computing-in-Memory Macro Supporting Wide-Range Dynamic Voltage-Frequency Scaling and Simultaneous MAC and Write Operations," in *2022 IEEE International Solid- State Circuits Conference (ISSCC)*, 20-26 Feb. 2022 2022, vol. 65, pp. 1-3.

[15] Z. Xuan, C. Liu, Y. Zhang, Y. Li, and Y. Kang, "A Brain-Inspired ADC-Free SRAM-Based In-Memory Computing Macro With High-Precision MAC for AI Application," *IEEE Transactions on Circuits and Systems II: Express Briefs,* vol. 70, no. 4, pp. 1276-1280, 2023.

[16] J. W. Su *et al.*, "Two-Way Transpose Multibit 6T SRAM Computing-in-Memory Macro for Inference-Training AI Edge Chips," *IEEE Journal of Solid-State Circuits,* vol. 57, no. 2, pp. 609-624, 2022.

[17] Y. Chen *et al.*, "SAMBA: Single-ADC Multi-Bit Accumulation Compute-in-Memory Using Nonlinearity- Compensated Fully Parallel Analog Adder Tree," *IEEE Transactions on Circuits and Systems I: Regular Papers,* vol. 70, no. 7, pp. 2762-2773, 2023.